FACULTY OF INFORMATION COMMUNICATION TECHNOLOGY (FICT)
PROG102-PRINCIPLES OF PROGRAMMING LOGIC AND DESIGN

Title	: Designing Logical Solutions for Sierra Leone Community Challenges: Flood Early Warning Alert System
Issue Date	: Week 2
Due Date	: Week 4
Lecturer/Examiner	: Mr Elijah Fullah.
Name of Student	: Edward Henry Musa
Student ID No.	: 905006526
Class	: BSEM1102
Semester/Year	: 1/1

Academic Honesty Policy Statement

I hereby attest that contents of this attachment are my own work. Referenced works, articles, art, programs, papers or parts thereof are acknowledged at the end of this paper. This includes data excerpted from CD-ROMs, the Internet, other private networks, and other people's disk of the computer system.

Student's Signature: Date:

LECTURER'S COMMMENTS/GRADE:

for office use only upon receive
Remark

DATE :
TIME :
RECEIVER'S NAME :

Table of Contents

1. Rationale **3**
a) Problem Description 3
b) Background 3
c) Community Impact 3
d) Proposed Solution 3
e) Computational Thinking Process 3
f) Algorithm Design 4
g) SDG Alignment 4
h) Open Source Justification 4
i) Digital Public Goods Justification 4
2. Algorithm **5**
3. Pseudocode **6**
4. Flowcharts **8**
5. Functional Modules **10**
6. Expected Outputs **12**
7. GitHub Evidence **13**
8. References **14**

1. Rationale

a) Problem Description

Every rainy season, flooding threatens communities across Sierra Leone, especially in Freetown and other fast-growing towns. Families living in low-lying and informal settlements often receive little or no warning before water enters their homes. The problem this project addresses is the **lack of a fast, reliable and simple system that turns rainfall and river-level readings into clear flood warnings for the people at risk.**

Who is affected: residents of flood-prone neighbourhoods, market traders, school children, farmers near rivers, and disaster response workers. **Why the problem exists:** heavy seasonal rainfall, blocked drains, building on flood plains, deforestation on hills, and warning methods that reach people late or only reach those with smartphones or radios.

b) Background

Sierra Leone has one of the highest rainfall levels in West Africa, with the heaviest rains from July to September. Freetown sits between steep hills and the sea, so runoff moves quickly into densely settled areas. The August 2017 flood and mudslide in the Regent area of Freetown, which reportedly killed more than 1,000 people, showed how costly late warnings can be. Mobile phones are widely used in Sierra Leone, and basic phones can receive SMS without internet access. This makes SMS a practical channel for alerts. Before any software is built, a Junior System Analyst must first design the logic of how such a system should think and act, which is the purpose of this assignment.

c) Community Impact

- **Problem being solved:** late and unclear flood warnings.
- **Expected impact:** earlier evacuation, fewer injuries and deaths, protected homes and livelihoods, and better records for planners.
- **Beneficiaries:** residents in flood-prone areas, schools, traders, farmers, local councils, and emergency responders.
- **Long-term sustainability:** the system uses low-cost SMS, can be run by trained community volunteers and local authorities, and its open design lets local developers maintain and improve it.

d) Proposed Solution

The proposed solution is a **Flood Early Warning Alert System**. An authorised officer or community volunteer enters the location, rainfall (mm) and river level (m). The system validates the readings, compares them with safe thresholds, classifies the flood risk as Low, Medium or High, builds a suitable message, and sends it by SMS to every registered contact in that location. Each check is recorded in a log. The design is divided into six modules so each part is simple to test and improve.

e) Computational Thinking Process

- **Decomposition:** the problem is broken into six smaller tasks: get input, validate, calculate risk, generate alert, send alert and log.
- **Pattern recognition:** floods follow repeating patterns. High rainfall combined with a high river level is the signal that most often comes before flooding, and this repeats every rainy season.
- **Abstraction:** the design focuses only on what matters for the decision (location, rainfall, river level) and ignores details such as the type of sensor or phone network.
- **Algorithm design:** the steps are arranged into a clear sequence with decisions and loops, shown in the algorithm, pseudocode and flowcharts.

f) Algorithm Design

The algorithm reads the data, checks that it is valid, uses Boolean logic and decision structures to classify risk, and uses repetition structures to send alerts to each contact and to keep checking new locations. The complete step-by-step algorithm is given in Section 2, the pseudocode in Section 3, and the flowcharts in Section 4.

g) SDG Alignment

- **SDG 11: Sustainable Cities and Communities.** Target 11.5 aims to reduce deaths and losses caused by disasters, including water-related disasters. Early warnings directly support this.
- **SDG 13: Climate Action.** Target 13.1 aims to strengthen resilience and the capacity to adapt to climate-related hazards. Flood alerts help communities adapt to heavier rainfall.
- **SDG 3: Good Health and Well-being** is supported indirectly, because fewer flood injuries and less contaminated water reduce disease risk.

h) Open Source Justification

The system should be **open source** so that its logic can be inspected, trusted and improved by anyone. For Sierra Leone, this means no licence fees, freedom from dependence on one vendor, and the chance for local universities, developers and councils to adapt it to their own districts and languages. Community members could contribute translations of alert messages, better threshold values, new alert channels such as radio or WhatsApp, bug fixes, and documentation. Existing open source tools such as FrontlineSMS and Ushahidi show that open tools can work well for community messaging and crisis reporting.

i) Digital Public Goods Justification

A Digital Public Good (DPG) is open source software, open data or open content that is free to use, adapt and share, and that supports the SDGs. This system could become a DPG by publishing its code and design under an open licence, documenting it clearly, protecting user privacy, and doing no harm. It promotes **accessibility** by working on basic phones through SMS, **openness** by allowing anyone to reuse and adapt it, and **public benefit** by giving life-saving information freely to every community. DHIS2, a well-known DPG used in health systems, shows that this model can work for public services.

2. Algorithm

The following is the step-by-step algorithm for the Flood Early Warning Alert System.

1. Start.
2. Define the constants RAIN_LIMIT = 50, RIVER_WARNING = 3.5 and RIVER_DANGER = 5.0.
3. Input the location, rainfall and river level.
4. Validate the readings. If either reading is negative, display "Invalid reading" and return to step 3.
5. Calculate the risk level (Low, Medium or High) using the thresholds.
6. If the risk level is Low, display "No alert needed" and go to step 9.
7. Generate the alert message for the location and risk level.
8. For each registered contact in the location, send the alert message by SMS.
9. Record the location, risk level and time in the log.
10. Ask whether another location should be checked. If yes, return to step 3.
11. End.

3. Pseudocode

```
START FloodEarlyWarning

CONSTANT RAIN_LIMIT = 50
CONSTANT RIVER_WARNING = 3.5
CONSTANT RIVER_DANGER = 5.0

DECLARE location AS String
DECLARE rainfall, riverLevel AS Decimal
DECLARE riskLevel, message AS String
DECLARE continueCheck AS Boolean

SET continueCheck = TRUE

WHILE continueCheck = TRUE
    CALL getInput(location, rainfall, riverLevel)
    IF validateData(rainfall, riverLevel) = FALSE THEN
        DISPLAY "Invalid reading. Try again."
    ELSE
        SET riskLevel = calculateRisk(rainfall, riverLevel)
        IF riskLevel = "Low" THEN
            DISPLAY "No alert needed."
        ELSE
            SET message = generateAlert(location, riskLevel)
            CALL sendAlert(location, message)
        END IF
        CALL logAlert(location, riskLevel)
    END IF
    DISPLAY "Check another location? (Y/N)"
    INPUT answer
    IF answer = "N" THEN
        SET continueCheck = FALSE
    END IF
END WHILE

END FloodEarlyWarning


FUNCTION validateData(rainfall, riverLevel)
    IF rainfall < 0 OR riverLevel < 0 THEN
        RETURN FALSE
    END IF
    RETURN TRUE
END FUNCTION


FUNCTION calculateRisk(rainfall, riverLevel)
    IF riverLevel >= RIVER_DANGER OR (rainfall > RAIN_LIMIT AND riverLevel >=
    RIVER_WARNING) THEN
        RETURN "High"
```

```
    ELSE IF riverLevel >= RIVER_WARNING OR rainfall > RAIN_LIMIT THEN
        RETURN "Medium"
    ELSE
        RETURN "Low"
    END IF
END FUNCTION


FUNCTION generateAlert(location, riskLevel)
    IF riskLevel = "High" THEN
        RETURN "DANGER: Flooding likely in " + location + ". Evacuate now."
    ELSE
        RETURN "WARNING: Flood risk rising in " + location + ". Stay alert."
    END IF
END FUNCTION


FUNCTION sendAlert(location, message)
    FOR EACH contact IN registeredContacts(location)
        SEND SMS message TO contact
    END FOR
END FUNCTION


FUNCTION logAlert(location, riskLevel)
    SAVE location, riskLevel, currentDateTime TO alertLog
END FUNCTION
```

4. Flowcharts

Flowcharts were created with Graphviz. Rounded boxes are start and end points, rectangles are processes, parallelograms are input or output, and diamonds are decisions.

Figure 1: Main program flowchart

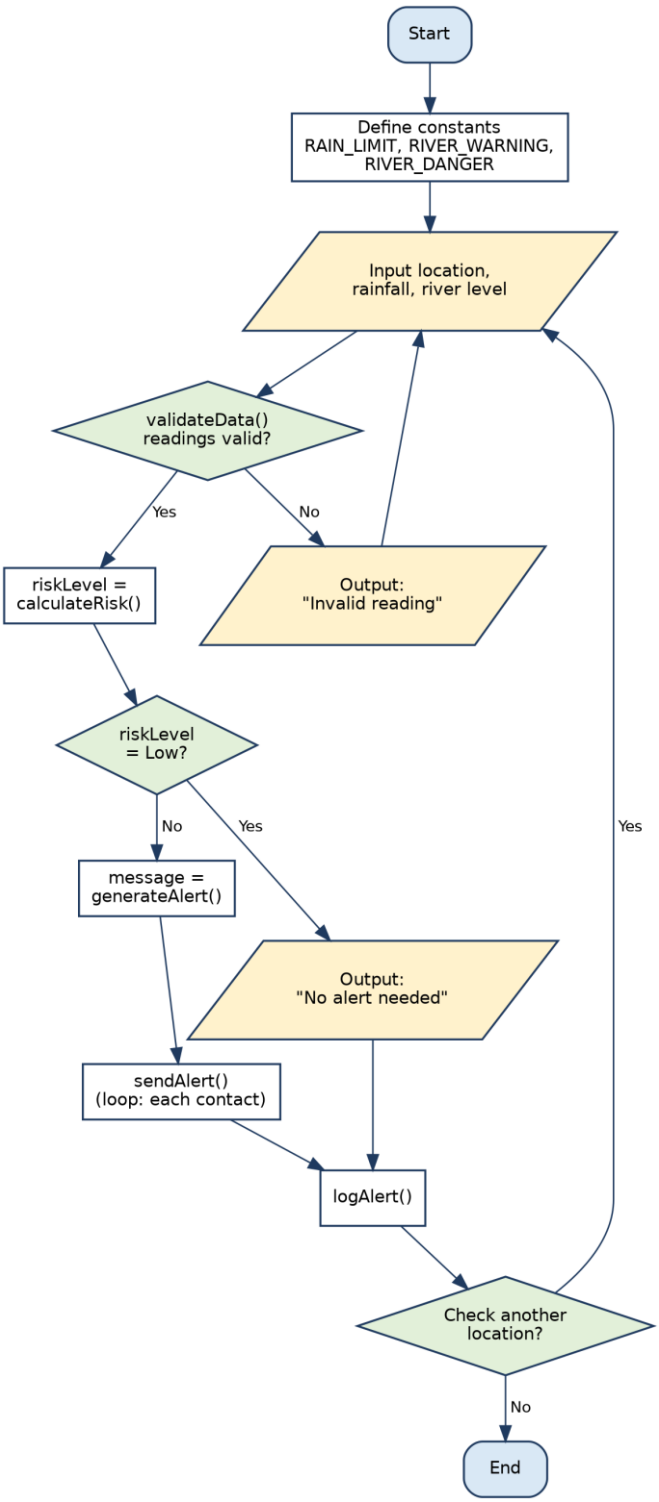


Figure 2: calculateRisk() function flowchart

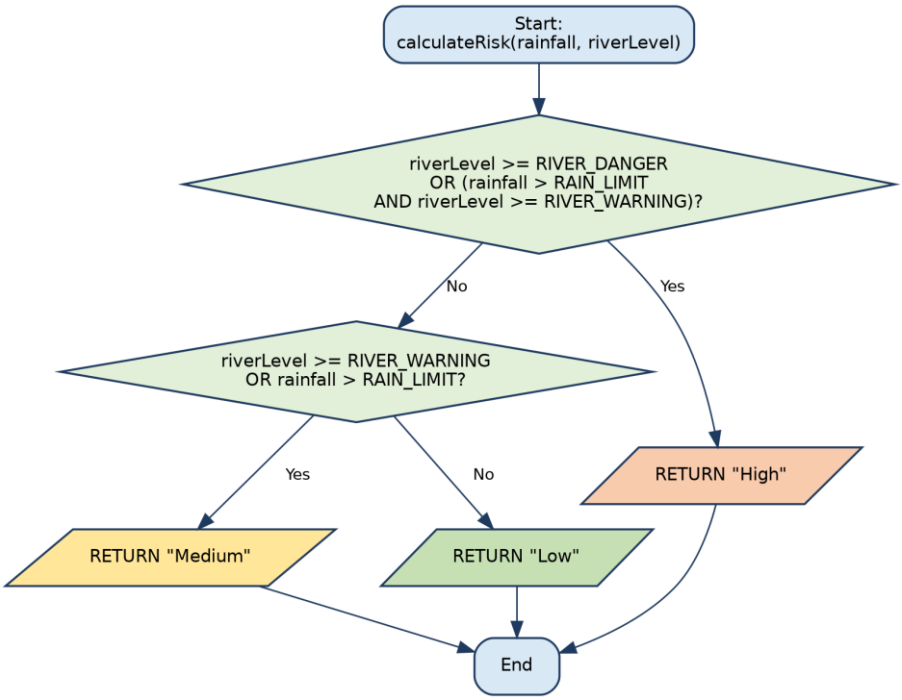
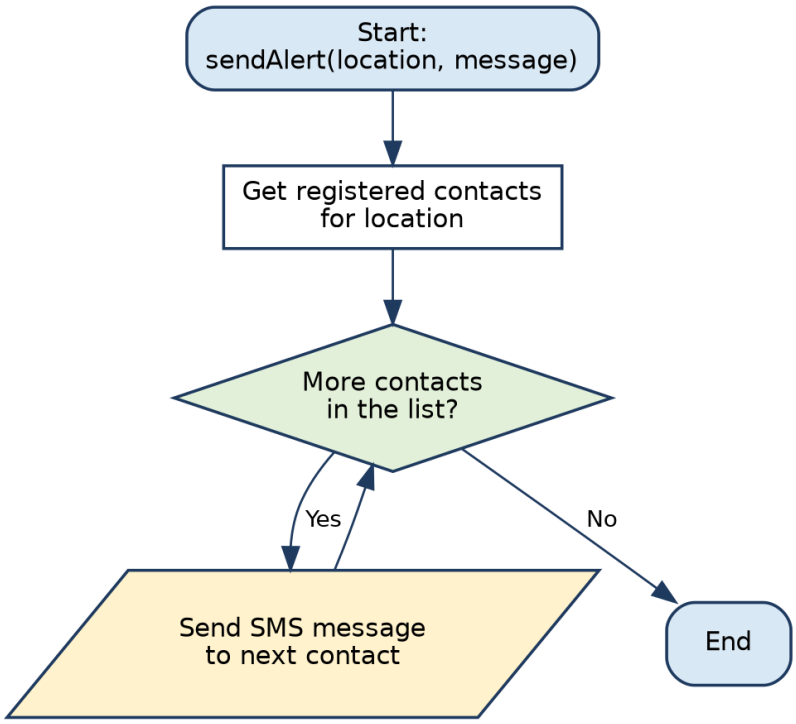


Figure 3: sendAlert() repetition flowchart



5. Functional Modules

Modular design divides the program into functions that each do one job. This makes the logic easier to test, reuse and maintain.

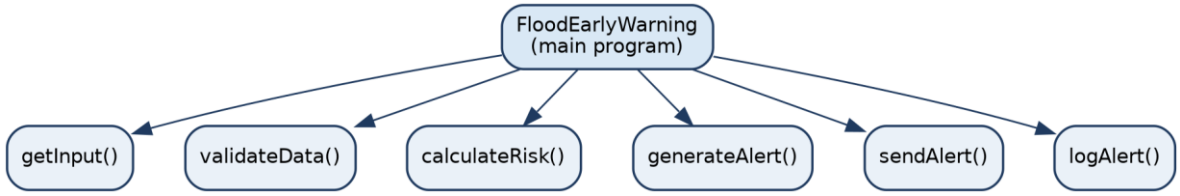


Figure 4: Module structure

Module	Input / Output	Purpose
getInput()	Returns location, rainfall, riverLevel	Collects the readings from the officer or volunteer.
validateData()	Takes rainfall, riverLevel; returns Boolean	Rejects negative or missing readings.
calculateRisk()	Takes rainfall, riverLevel; returns String	Uses Boolean logic and decisions to return Low, Medium or High.
generateAlert()	Takes location, riskLevel; returns String	Builds the correct message for Medium or High risk.
sendAlert()	Takes location, message	Uses a loop to send an SMS to every registered contact.
logAlert()	Takes location, riskLevel	Saves a record of every check for review.

Constants

Constant	Value	Meaning
RAIN_LIMIT	50 (mm)	Rainfall above this is considered heavy.
RIVER_WARNING	3.5 (m)	River level at which risk begins to rise.
RIVER_DANGER	5.0 (m)	River level at which flooding is very likely.

These values are examples for this design. In a real deployment they would be set with advice from the Sierra Leone Meteorological Agency and the National Disaster Management Agency.

Variables and data types

Variable	Data type	Purpose
location	String	Name of the community or area being checked.
rainfall	Decimal	Rainfall reading in millimetres.
riverLevel	Decimal	River level reading in metres.
riskLevel	String	Result of calculateRisk(): Low, Medium or High.

Variable	Data type	Purpose
message	String	The alert text sent to contacts.
continueCheck	Boolean	TRUE while the user wants to check more locations.

6. Expected Outputs

The table below shows the expected output for sample scenarios.

No.	Rainfall (mm)	River (m)	Risk	Expected output
1	20	2.0	Low	No alert needed.
2	30	4.0	Medium	SMS: "WARNING: Flood risk rising in [location]. Stay alert."
3	70	2.0	Medium	SMS: "WARNING: Flood risk rising in [location]. Stay alert."
4	70	4.0	High	SMS: "DANGER: Flooding likely in [location]. Evacuate now."
5	10	5.2	High	SMS: "DANGER: Flooding likely in [location]. Evacuate now."
6	-5	2.0	Invalid	Invalid reading. Try again.

7. GitHub Evidence

Replace each box below with the matching screenshot or text from your own repository.

Item	Evidence
GitHub repository name	PROG102_Assignment1_EdwardHenryMusa_905006526
Repository URL	https://github.com/edwardhenry552/PROG102_Assignment1_EdwardHenryMusa_905006526
Git commit history	[Insert screenshot of the commit history page]
README.md	[Insert screenshot of README.md]
Repository screenshot	[Insert screenshot of the repository main page showing all files]

8. References

United Nations (n.d.) The 17 Sustainable Development Goals. Available at: https://sdgs.un.org/goals (Accessed: November 2026).

Digital Public Goods Alliance (n.d.) What are digital public goods? Available at: https://www.digitalpublicgoods.net (Accessed: November 2026).

Open Source Initiative (n.d.) The Open Source Definition. Available at: https://opensource.org/osd (Accessed: November 2026).

GitHub (n.d.) GitHub Docs. Available at: https://docs.github.com (Accessed: November 2026).

Limkokwing University of Creative Technology, Sierra Leone (2026) PROG102 Assignment 1 brief: Designing Logical Solutions for Sierra Leone Community Challenges.